

said, "I asked him to go through it, format the machine and start installing. They started installing but a major issue that came up was that a few things that were obvious to me were not as obvious to them. In the beginning, I may have taken a few things for granted, but when the team actually started installing, I found gaps in my documentation. I improved on it and only when my documentation was thorough, I started distributing it across my team. I asked team members to install Ubuntu on at least three-four machines and report their experiences to me on a regular basis. After that, we encouraged a small team that operates from our Tardeo office in Mumbai. I told them that I would give them different software and ensured support to them in case they faced any issues. Initially I gave them their existing machines as well but asked them to refrain from touching them." Joel used the findings of this sample set-up to improvise on his documentation for setting up further models.

The activity of setting up the new office began in January. Since all the 75 desktops Joel had to set up required a similar configuration, he got one system ready and then used Clonezilla to clone this system onto the remaining 74 systems.

Key benefits to the organisation

- Reduced TCO, which is crucial for a growing organisation
- Easy integration of the Ubuntu system in an existing environment
- Secured environment
- Better systems management
- Reduced downtime
- Vast online community support

Joel replicated the same system in other units of his company and continues to do so even today. He said, "I did not have to put in much effort to convince the management because of the value that open source offers. Not only did it ensure cost benefits, but also technology benefits." 

By: Diksha P Gupta

The author is assistant editor at EFY. When she is not exercising her journalistic skills, she spends time in travelling, reading fiction and biographies.

Continued from Page 73...

- Low level of maintenance: An MQ system should not demand too much maintenance, especially if your team is small, because your actual work is not to maintain queues—it is to develop your app.
- Ease of deployment: Deployment should be easy, and it should not alter your existing deployment process too much. It should be easy for you to add the MQ in your stack.
- Durability and persistence: Message queues generally store messages in the main memory, and that means that your messages are vulnerable to getting lost if something goes wrong. According to your application, and the importance of your messages, you might want to consider how this is handled. If your broker goes down, is it able to reconstruct queues with minimum loss when it comes up? When the complete system goes down, since the messages are stored in main memory, what happens when the system comes up? Does it have any level of persistence? What level of reliability does your broker offer? Depending on your application, these questions take on different levels of importance.
- Community support: This can be an important factor. Community support can be the life line when you get stuck. An active community will always be helpful while dealing with different kinds of problems.
- Cluster support: Can multiple broker installations on a network be clustered together? Again, this is very use-case specific.

How to put it all together

So, we have taken a good look at how to make use of message queues. In production, a general set-up would look something like Figure 3. The Web server catering to your application's traffic can be on one server and your database on another server. Your task manager (like Celery) can be on the same server as the Web application. Your broker should be on a different server, so that if something goes wrong with it, your Web server remains safe, and vice-versa. Your workers can be either on the server where the broker is, on a different server, or even on multiple different servers, depending on (for example) the number of tasks or the nature of tasks.

Building usable and responsive Web applications is important. We saw how queues can help us with improving responsiveness and looked at a couple of use-cases where they can be helpful. But the possibilities are endless, not only in Web applications, but almost anywhere. We also saw that it's not all that difficult to set up queues. You can make use of queues with simple libraries such as PyRes, or something much more robust like Celery, in just about no time. I hope you found this tutorial useful, and I hope you are already on your way to building some amazing apps. 

By: Vaidik Kapoor

The author is interested in distributed computing, Web applications, scalable systems and in building anything that solves real problems. He is a FOSS enthusiast and a foodie. He blogs at <http://vaidikkapoor.info>.